University of Mons
Sciences Faculty
Computer Science Department

LoRa frame detection and capture
Internship Report

Head of Service :
Bruno Quoitin

Supervisor :
Aqeel Ahmed

Author :
Maxime Bartha

Academic Year 2024-2025

Contents

1 Introduction **2**
1.1 Organization of the Report 2
1.2 Objective 2

2 LoRa Basics **3**
2.1 Chirp Spread Spectrum modulation 3
2.2 Preamble 4
2.3 Demodulation 5

3 Methodology **6**
3.1 Experimental Setup 6
3.2 Transmission method 9
3.3 Scenarios 10

4 Implementation **11**
4.1 Initiation 12
4.2 Transmission 12
4.3 Termination 12
4.4 Error Handling 12

5 Exploratory data analysis **13**
5.1 Data overview 16

6 Conclusion **17**
6.1 Possible improvements 17

Appendices **20**

A First appendix **20**

B Second appendix **20**

1 Introduction

With the rapid growth of Internet-of-Things (IoT) deployment for several daily life applications, the security of these devices is a key concern. Especially when the existing cryptography-based security techniques are resource-hungry and still vulnerable. Therefore, innovative and robust security mechanisms are urgently needed to protect the IoT end devices from malicious actors.

Radio Frequency Fingerprint Identification (RFF) is an emerging technology and has been seen as a key candidate for device authentication at the physical layer [1]. The RF fingerprints are extracted from the received radio signal impaired by the transmitter's hardware components. These impairments are subtle, though unique to each transmitter, making them useful for device identification. However, manual extraction of these features is challenging and time-consuming. Therefore, Deep Learning (DL) based RFFI has been proposed as a vital solution for this field. Although DL based RFFI has shown enormous potential for emitter identification, but these algorithms require a massive amount of training. Availability of quality radio signal data is a key challenge in this domain [1].

This report presents our efforts to collect Long-range (LoRa) RF signal data during the one-month BIR-supported summer internship period from July to August 2025. We systematically established test-bed scenarios in the indoor and outdoor environments to collect diverse LoRa RF signal data.

1.1 Organization of the Report

Section 2 explains the basic knowledge required to understand LoRa related discussions in this report.
Section 3 presents the equipment used, the structure of experiments as well as the different scenarios tested. A discussion on the possible synchronization strategy is also made to assess the most reliable method.
A description of the selected implementation is made in section 4.
The results and their quality are assessed and discussed in section 5.
Finally, in section 6, propositions of improvements are presented and the conclusion of this work is done.

1.2 Objective

The internship aimed to build an understanding of the LoRa technology, RF fingerprinting as physical layer security, and the applications of deep learning for LoRa RF Fingerprinting. The key objectives are listed below:

- **Understand the working of LoRa technology**
- **Understand the applications of deep learning for LoRa RF fingerprinting**
- **Establish a test bed scenario to collect LoRa RF signals in indoor and outdoor environments**.
- **Collect LoRa RF signals at different distances to evaluate the DL model's performance under an unseen long-distance scenario**.

2 LoRa Basics

In this section, the essentials of the LoRa modulation technique (section 2.1), preamble format (section 2.2) and demodulation technique (section 2.3) are discussed to understand their uses in the experiments.

2.1 Chirp Spread Spectrum modulation

LoRa is a long-range, low-power, proprietary radio interface. It is based on a Chirp Spread Spectrum (CSS) modulation technique. The basic element of this technique is called a chirp. A chirp is a sinusoidal signal whose frequency varies linearly over time. Details on the underlying math can be found in a recent article [3] explaining CSS modulation and the key advances made to understand it. To understand the experiments made in this paper, only the frequency domain is explored and explained. The Figure 1 represents briefly the relation between the frequency and time domain.

There are three necessary parameters for LoRa communication : a carrier frequency (F_c), a bandwidth (B) and spreading factor (SF).
The carrier frequency of the signal is set in Europe as 868 MHz. Finally, the spreading factor represents the number of bits carried in a LoRa symbol. For example, with an SF of 7, a LoRa symbol can represent a value ranging from 0 to $2^7 - 1 = 127$.

LoRa symbols are chrips whose duration T depends on the bandwidth B and spreading factor SF as the equation 1 shows.

$$T = \frac{2^{SF}}{B} \tag{1}$$

A LoRa symbol frequency varies from $F_c - B/2$ to $F_c + B/2$ linearly as shown in the Figure 2a. It implies that when a signal reaches a frequency of $F_c + B/2$

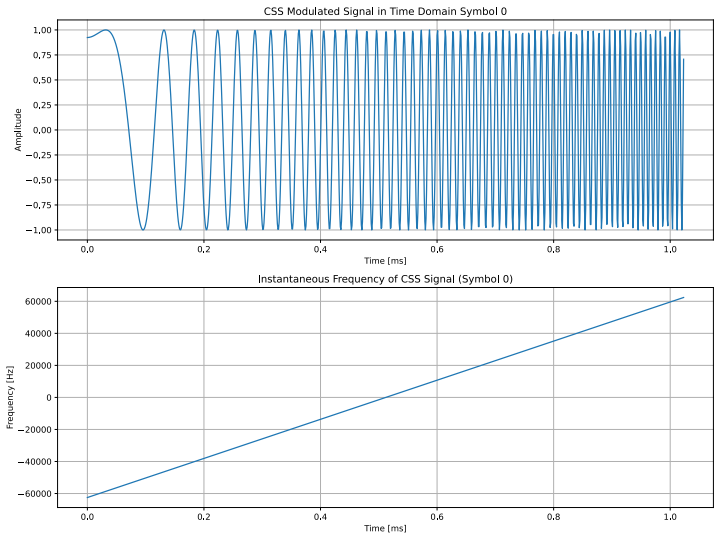


Figure 1: Representation of a LoRa symbol in the time domain (Top) and the frequency domain (bottom).

(resp. $F_c - B/2$), wraps up to $F_c - B/2$ (resp. $F_c + B/2$) before varying linearly again.

The value represented depends on the start (and final) frequency of the LoRa symbol. For a symbol of value s, the starting frequency $f_0 = F_c - B/2 + \frac{sB}{2^{SF}}$. Intuitively, the value is represented by the "frequency phase" of the received signal. It is possible to observe that the symbol represented in Figure 2b is a baseup chirp because its starting frequency is $F_c - B/2$.
A baseup chirp represents a value of 0 and varies from $F_c - B/2$ to $F_c + B/2$. Similarly, a base-down chirp starts at $F_c + B/2$ and ends at $F_c - B/2$, decreasing linearly rather than increasing.

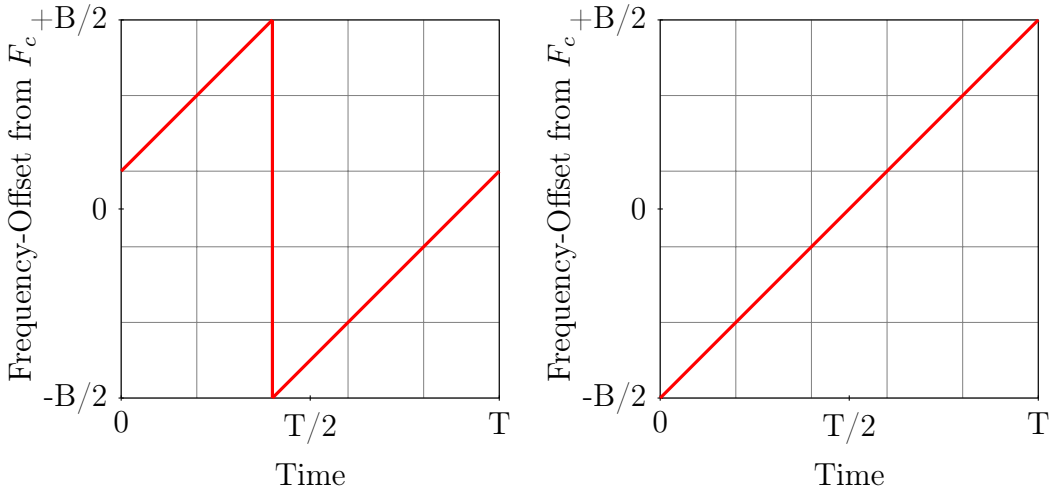
2.2 Preamble

A LoRa frame is composed of 3 parts: the preamble part, the Start of Frame Delimiter (SFD) and the actual data symbols. The preamble part is composed of 8 or more baseup chirps and a sync word part composed of two symbols. The SFD is composed of 2.25 base-down chirps.

A captured representation of a preamble + SFD can be seen in Figure 3.

The preamble consists of unmodulated, identical base-up chirps generated

[1]inspectrum is an open source radio signal analyser. Its source code can be found at `https://github.com/miek/inspectrum`



(a) LoRa Symbol

(b) baseup chirp

Figure 2: Chirps of duration T and bandwidth B in the frequency domain (frequency with respect to time). The frequency axis is relative to the carrier frequency F_c.

Figure 3: Spectrogram of a captured LoRa preamble and SFD from inspectrum[1]

directly from the transmitter's RF chain without data encoding. This makes it ideal for capturing hardware-specific distortions such as carrier frequency offset (CFO), I/Q imbalance, and phase noise—core features used in RF fingerprinting. The SFD is shorter and may not reveal stable and consistent features. Whereas the payload can vary in size, hence it cannot be considered suitable for DL based RFFI [5, 4].

2.3 Demodulation

The demodulation technique used in the experiments comes from a comprehensive paper named *From Demodulation to Decoding* [6]. The method proposed can be described in 4 steps graphically explained in Figure 4.

i. Retrieve the sampled data containing a LoRa symbol with a sampling frequency f_{samp} of at least two times the bandwidth Bw.

ii. Multiply the retrieved signal, of initial frequency f_0, with a base downchirp to obtain two constant signals with frequency f_0 and $f_0 - Bw$.

iii. Apply a Fast Fourier Transform (FFT) on the result of the multiplication. This gives two distinct peaks at f_0 and $f_{samp} - Bw + f_0$ as shown in Figure 4(iii).

iv. The paper then proposes two options to merge those peaks. A Fine-Grained Phase Alignment (FPA) and a Coarse-grained Phase Alignment. CPA is much less computationally heavy than FPA and has marginally worst results. In either case, the result gives an FFT peak at f_0.

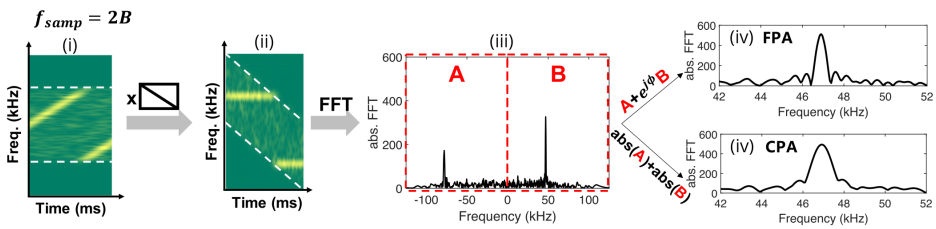


Figure 4: Graphical explanation used in the reference paper [6].

3 Methodology

The methodology section covers the hardware used, the test-bed established (Section 3.1), the transmission method of the frames the transmitter to the receiver (Section 3.2) and the scenarios of the experiments (Section 3.3).

3.1 Experimental Setup

We used Arduino-based MKR-WAN 1310 LoRa transmitters for sending LoRa frames and USRP B210 software-defined radio (SDR) for the capture. The MKR-WAN 1310 transmitters were attached with Molex 868/915 LoRa compatible antenna, while the USRP B210 was attached with commonly used whip antennas. It was ensured that the polarization between the transmitter and receiver remains the same (vertical). Each device was fixed at least 10 cm apart from each other to avoid contact between the antennas and cause interference. The transmitters were powered through an external supply (Elleman 5V lab power supply (LABPS3005SM)) to maintain a stable voltage and current supply for all the transmitters. All the transmitters were

connected to a single laptop through 4 StarTech USB hubs (ST7200USBM). On the receiver side, the USRP B210 SDR was connected to the receiving end laptop, which captured and processed the frames.

The receiver setup is straight forward : the laptop is connected to the USRP SDR by USB. The SDR antenna is vertical. Figure 5 shows a schematic of the receiver setup.

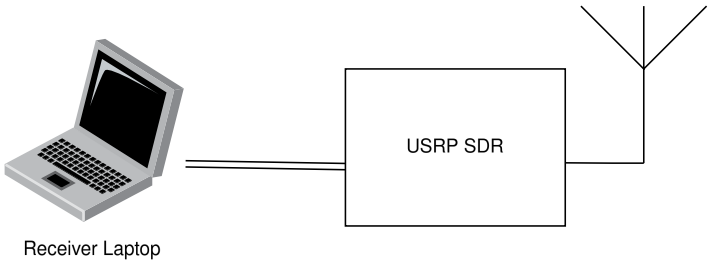


Figure 5: Receiver side schematic. The receiver laptop is connected to the USRP by USB. The SDR captures signals through its antenna.

photo of the Receiver part in appendix

The transmitter setup is more complex : the laptop is connected to a first USB HUB on which the 3 other USB hubs are connected to form a star topology. The choice of such a topology is to flatten the USB communication delay with all 20 Arduino devices. The Arduino devices are connected to the outermost USB hubs and powered externally by the lab power supply through their back pins with the soldered cables. The antennas connected to the devices are also vertical to have the same polarization as the SDR's antenna.
The power supply is programmed with 5 V of tension and 1 A of current.
The Figure 6 shows the USB connections from the laptop to the Arduino devices. Figure 7 shows the power supply schematic of the transmitter side.

photo of the transmitter part in appendix

add a reference for the polarization effect

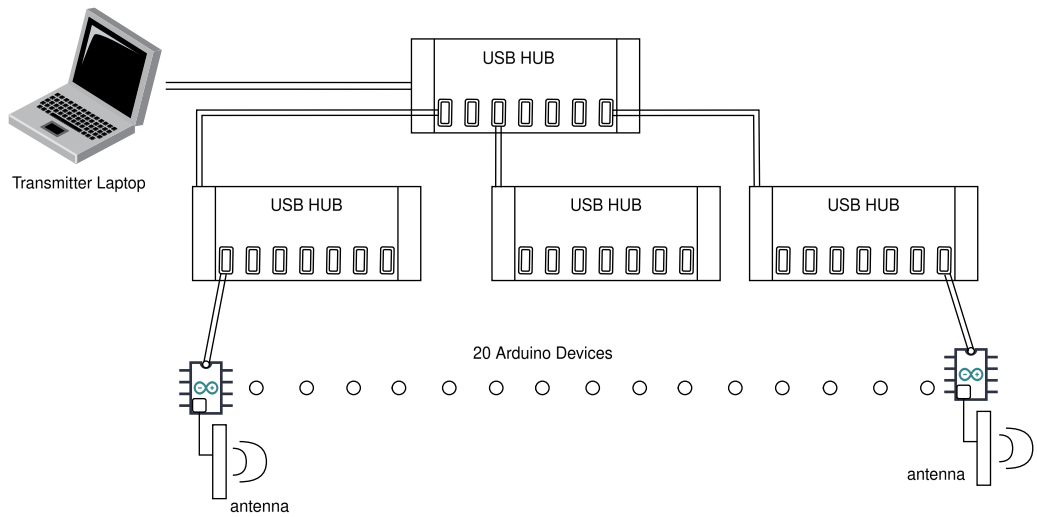


Figure 6: Receiver side USB connections schematic. The transmitter laptop is connected to the top level USB hub on which the outermost hubs are connected. Twenty devices with antennas are then connected to the outermost hubs.

The layout of an experiment has tree components : The receiver and transmitter part composed of the receiver and transmitter setup respectively and a smartphone used as a modem for the communications between them. All of those components are in line of sight of each other to have the best possible transmission, reception and communication. Figure 8 shows a representation of the layout.

Add photos of the layout in appendix

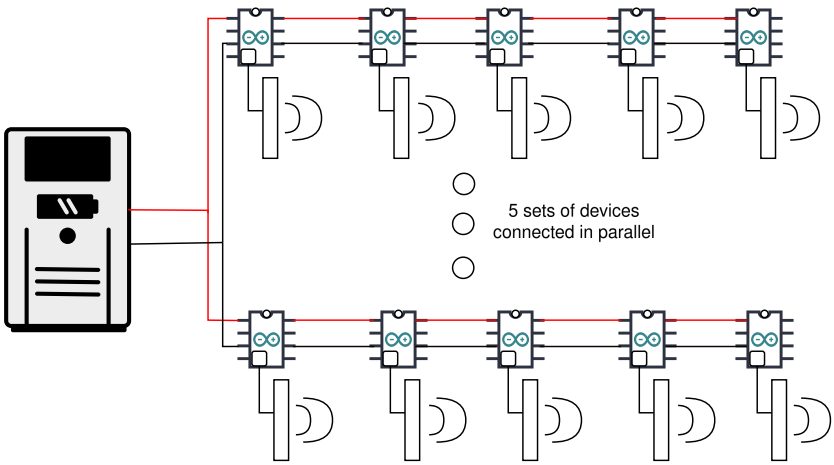


Figure 7: Receiver side power supply schematic. The composition is as follows : five sets of devices are connected in parallel to the lab power supply. In a set all devices are connected in series through their back pins (VIN and GND).

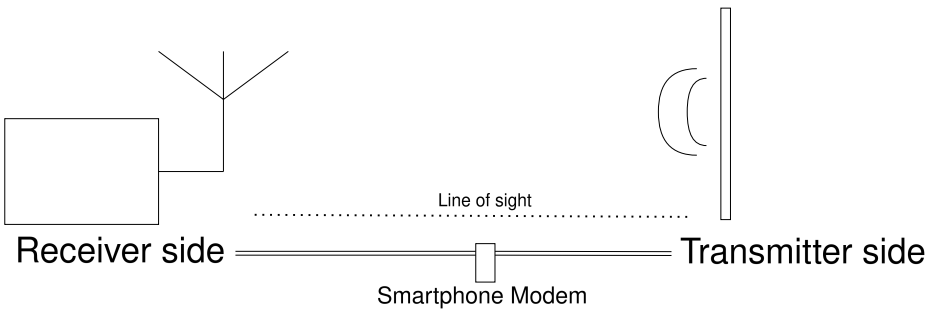


Figure 8: Representation of an experiment layout. All components have line of sight with the others. The smartphone is used as a connection bridge between the receiver and transmitter side.

3.2 Transmission method

The method to transmit and receive frames is discussed with two proposed methods. In each of them, the devices take turns one after the others. The difference is how each device handles its turn.

Round-robin method (RR) : on its turn, a device transmits a LoRa frame, then passes the turn to the next one. This is done in a cycling manner, meaning the last device gives the turn to the first one. The method's

transmissions goes on until every frame from every device is sent.

One by One method (ObO) : on its turn, a device transmits every LoRa frames it has to, then passes the turn to the next one. Hence, no cycles are needed because at the end of the last device's turn, the transmissions are finished.

Comparison : the methods are compared by (1) speed, (2) ease of implementation, (3) misclassification probability and (4) quality. A misclassification occurs when the receiver labels a frame as coming from a device, although in reality, it's coming from another device. Misclassifications cannot occur, otherwise the database is unusable without finding and removing them. The quality metric refers to the uniformity of the captured frames.

(1) Each method is similar in terms of speed, though ObO can be faster because there is no increase in the possible misclassifications as speed increases.

(2) Both implementations have their specificities, but none are substantially harder to implement than the other.

(3) Misclassifications are most likely to occur when changing devices. The number of changes in RR makes it prone to misclassifications. In contrast, ObO has way fewer device changes, making it less prone to misclassification.

(4) The environment can vary over time (temperature, humidity, ...) and the Deep Learning model is capable of identifying those variations and classify devices with respect to them.
In RR, those variations are present on all devices uniformly because they take turns quickly.
On the other hand, in ObO, those variations aren't present uniformly across all devices because the time between each device is much longer. For example, between the transmissions of the first and last device, all the other devices sent their frames, which takes a considerable amount of time. This elapsed time can imply different variations between the first and last device, making the deep learning model capable of more easily distinguishing between them.

Selection : although the quality of the captures from ObO can be lower than in RR, because of the lack of time and difficulty to make the Round-robin method work flawlessly, the One by One method is used in the experiments.

3.3 Scenarios

Captures are made in two environments at different distances. Some parameters are shared between each scenario. They are : the spreading factor (SF), bandwidth (Bw), carrier frequency (F_c), time between 2 frame transmissions ($period$), number of frames a device has to send, number of devices, gain on

the receiver SDR and sampling rate (F_s).
The values of the shared parameters are shown in Table 1.

Table 1: Shared parameters values

SF	**Bw** (kHz)	$\mathbf{F_c}$ (MHz)	**period** (ms)
7	125	868	200
frames per device	**number of devices**	**Gain** (dB)	$\mathbf{F_s}$ (samples/s)
550	20	+40	10^6

The period choice is a trade-off between speed and safety, going too fast can cause transmission issues.
The number of frames per device is also a trade-off between time and quantity. The gain is selected for the frames to be easily detected at longer distances. A higher sampling frequency is better for Deep Learning training, hence the choice of 10^6 samples per second. The SF, Bw, F_c are arbitrary radio frequency parameters fixed throughout the experiments.

The two environments used in the experiments are :

INDOOR : The sports hall in the Vésale building on the Nimy Campus at Umons is used as the inside environment. The distance varies from 5 m to 40 m with an initial step of 5 m then steps of 10 m until 40 m. Thus, the resulting captures are at 5 m, 10 m, 20 m, 30 m and 40 m.

OUTDOOR : The grass field between the DeVinci building and the "bâtiment 4" on the Nimy Campus at Umons is used as the outside environment. The distance varies from 5 m to 50 m with an initial step of 5 m then steps of 10 m until 50 m. Thus, the resulting captures are at 5 m, 10 m, 20 m, 30 m, 40 m and 50 m.

4 Implementation

The implementation stretches over 3 devices : the receiver laptop, the transmitter laptop and the Arduino devices. The objective of the implementation is to synchronize the transmission and reception such that the receiver is sure of the origin of each frame to correctly label it.

The overall interactions of each device are presented. For more details the implementation is available on GitHub at `https://github.com/Gnarks/BIR`. All the codes are upgraded versions of Maxime Bazin's 2024 implementation [2].

footnote or citation ?

The implementation has 3 main processes : initiation (Section 4.1), transmission (Section 4.2) and termination (Section 4.3). In addition, error han-

dling for unreachable Arduino devices is discussed in Section 4.4.

4.1 Initiation

The initiation process has 2 phases. First, the transmitter establishes communication with each Arduino device through USB and initiates them. Then, Once each device has acknowledged, the transmitter initiates the communication with the receiver, giving it the ordered list of devices, an initial time stamp to start the transmission process, and the period between two transmissions. For the second phase to be possible the receiver has to be in a waiting state before the initial communication.

4.2 Transmission

The transmission process is the main part of the implementation, it is in charge of the synchronization and labeling of the devices. After the initiation, each laptop starts at the shared time. Then each period, the transmitter laptop makes a device send a frame and the receiver labels it with respect of the number of periods elapsed since the starting time. Once every frame from a device are sent, the transmitter tells the receiver which device is sending its frames next with another start time. Such that, both the transmitter and receiver, at any time, know which device is currently transmitting and which frame is transmitted.
Figure 9a shows the general transmission scheme and Figure 9b shows the change from one device to another.

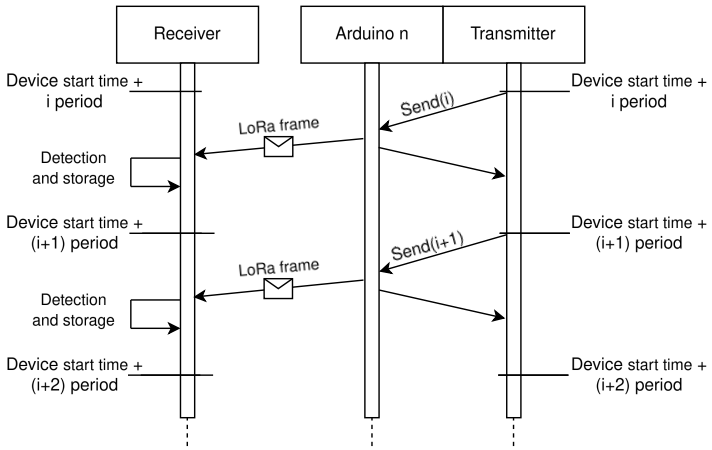
4.3 Termination

The termination process is the last step of the implementation. It occurs when every device sent every frame it had to. The transmitter waits a certain time before telling the receiver to stop the reception process and closes the communication.

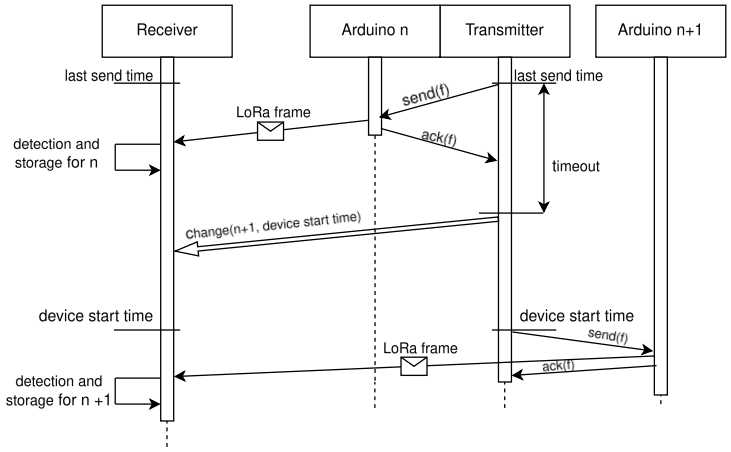
4.4 Error Handling

If for some reason during initiation or transmission, an Arduino device is unreachable it is removed it from list of devices. It is mandatory to remove the device from the list to avoid it to take time slots to send without transmitting anything.

An unreachable device can occur due to hardware or software issues, for example it is possible for a device to lose USB connection due the wind



(a) General transmission scheme. The transmissions and receptions are synchronized by sharing the device start time and a common period.



(b) Device change scheme. To pass the turn from a device n to a device $n+1$, the transmitter waits for a fixed timeout and communicates to the receiver the next device and the device start time.

Figure 9: Device transmission process represented by the general transmission scheme (a) and device change scheme (b).

causing a false contact. Software issues are less likely to occur but taken into account.

During initiation, if any device is unreachable, an error blocks the process. This prevents any false start and requires fixing the issue before restarting.

During transmission, if any device is unreachable, the transmitter stops then tells the receiver to remove this device from the list before continuing back where it needs after a shared time.

The process of removing a device from the transmission and reception list is irreversible, to avoid backtracking frames. It can be an issue if errors occur frequently, but in the case of our experiments, those errors rarely occurred. In addition, most of these errors occurred during the initiation step because of false contacts.

5 Exploratory data analysis

The outdoor and indoor scenarios are represented 2 and Table 3 respectively. They represent the number of captured frames per device and distance. The number in parentheses represent the double captures which are the files containing 2 frames instead of one. The double captures are not problematic in the case of the selected synchronization method (ObO), because it is possible

in post-processing to separate them. In the other method (RR) having double captures would have been devastating because the origin of the frames wouldn't be identifiable. One key observation to be made is the lack of clear decrease in frames captured with increasing distance.

Table 2: Frames Captured Indoor. The total number of frames is presented and in parentheses is the number of captures containing 2 frames.

Frames Captured indoor					
device ID	**5**m	**10**m	**20**m	**30**m	**40**m
1	539(0)	544(1)	541(0)	520(0)	537(0)
2	526(0)	542(0)	526(25)	530(0)	532(0)
3	543(0)	540(1)	541(121)	539(0)	535(0)
4	531(0)	528(0)	532(93)	530(1)	535(0)
5	533(0)	530(0)	531(52)	532(2)	536(0)
6	523(0)	531(1)	533(23)	527(4)	522(0)
7	533(0)	539(3)	544(20)	544(8)	540(0)
8	534(0)	530(9)	535(12)	542(22)	541(0)
9	542(0)	542(49)	530(14)	541(21)	540(0)
10	543(0)	543(47)	536(11)	541(28)	543(0)
11	540(0)	537(0)	542(11)	533(44)	533(0)
12	537(0)	539(0)	524(8)	533(48)	537(0)
13	525(0)	533(0)	534(4)	534(84)	527(0)
14	538(0)	535(0)	528(6)	539(102)	534(0)
15	524(0)	540(0)	509(3)	533(126)	519(0)
16	534(0)	537(0)	528(3)	535(130)	538(0)
17	541(0)	537(0)	531(3)	538(124)	526(0)
18	532(0)	535(0)	535(1)	533(132)	529(0)
19	538(0)	542(0)	541(3)	542(133)	540(1)
20	536(0)	534(0)	535(2)	537(112)	531(0)
total	10692	10738	10656	10703	10675

Table 3: Frames Captured Outdoor. The total number of frames is presented and in parentheses is the number of captures containing 2 frames.

Frames Captured outdoor						
device ID	**5**m	**10**m	**20**m	**30**m	**40**m	**50**m
1	520(95)	538(26)	532(2)	523(0)	535(56)	540(71)
2	531(124)	535(64)	532(18)	534(1)	534(32)	536(59)
3	534(37)	542(0)	536(7)	538(1)	536(10)	545(37)
4	532(2)	525(1)	532(1)	533(4)	535(12)	531(28)
5	466(0)	538(1)	537(3)	534(53)	538(45)	540(54)
6	517(0)	535(0)	533(67)	535(3)	529(76)	536(7)
7	530(2)	530(1)	533(16)	547(1)	541(44)	532(0)
8	482(0)	520(63)	523(3)	533(89)	536(13)	529(0)
9	514(1)	520(51)	537(2)	540(25)	540(0)	523(0)
10	528(1)	530(1)	541(8)	532(0)	541(0)	526(3)
11	476(1)	524(0)	541(2)	526(1)	546(0)	521(2)
12	505(0)	519(1)	539(2)	525(0)	542(4)	533(2)
13	508(1)	529(2)	528(0)	529(5)	535(6)	536(1)
14	476(63)	539(52)	541(2)	533(0)	537(5)	540(0)
15	503(47)	529(28)	542(3)	484(0)	530(1)	516(0)
16	529(11)	540(16)	532(1)	523(0)	533(96)	526(1)
17	527(2)	538(9)	530(5)	527(3)	537(55)	535(48)
18	529(2)	531(8)	529(8)	520(3)	540(1)	525(77)
19	526(2)	536(13)	540(1)	535(5)	545(3)	534(20)
20	527(1)	541(14)	530(27)	532(13)	539(0)	536(0)
total	10260	10639	10688	10583	10749	10640

5.1 Data overview

To get a better glimpse of the data, the following figures are presented for the 20th device's preamble indoor:

- Figure 10 : The time domain representation of the first frame at different distances.
- Figure 11 : The frequency domain representation of the first frame at different distances.
- Figure 12 : The constellation graph of the IQ samples at different distances.

In Figure 10, the relation between distance and amplitude is clearly shown. Between 20 m and 30 m the difference is less important than between 5 m and 10 m.

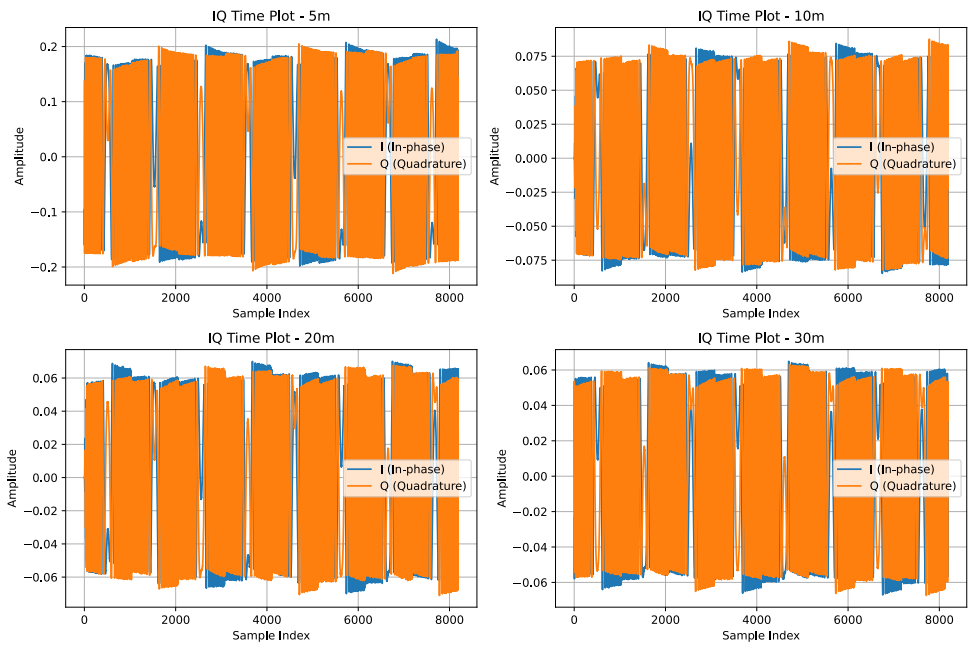


Figure 10: The IQ amplitude time plot of the 20th device preamble at different distances.

The spectrograms shown in Figure 11 represents the same preambles shown in Figure 10.

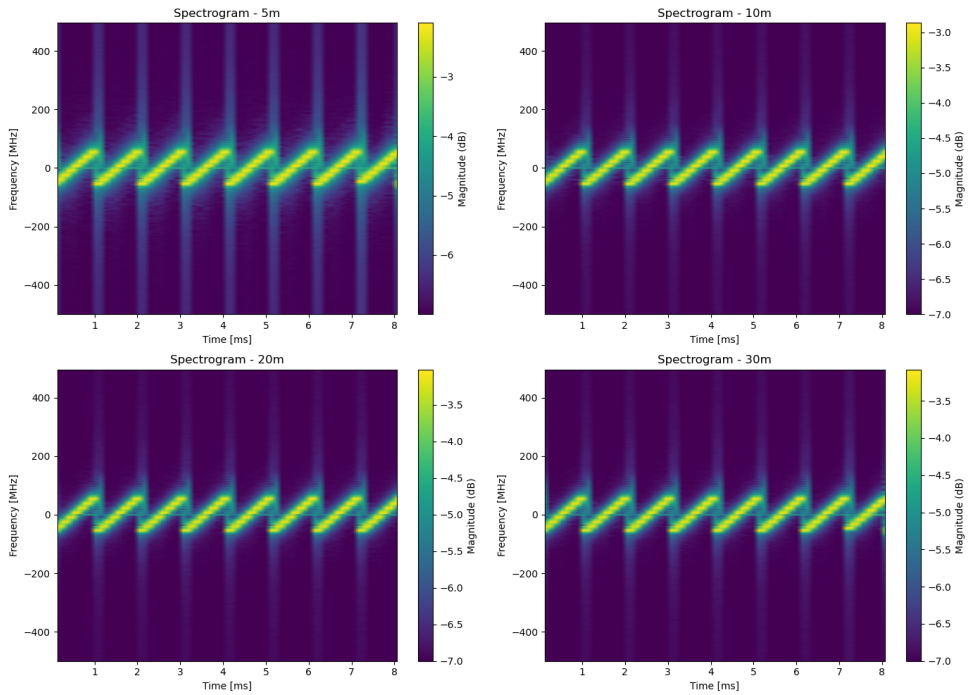


Figure 11: The spectrograms of the 20th device preamble at different distances.

6 Conclusion

As a conclusion, our efforts on collecting LoRa RF signal data during the internship as concluded in two datasets varying distance indoor and outdoor. Although the time of the internship was small, a lot of improvements are to be mentioned from last year experiments. Such as a more stable hardware structure without power issues and new synchronization methods.

6.1 Possible improvements

The possible improvements to be made in regard to this internship are the following :

1. Make the RR method work stably for better quality captures.
2. Not relying on a mobile phone modem for communication between laptops.
3. Fine-tune the detection algorithm to make it faster and safer.

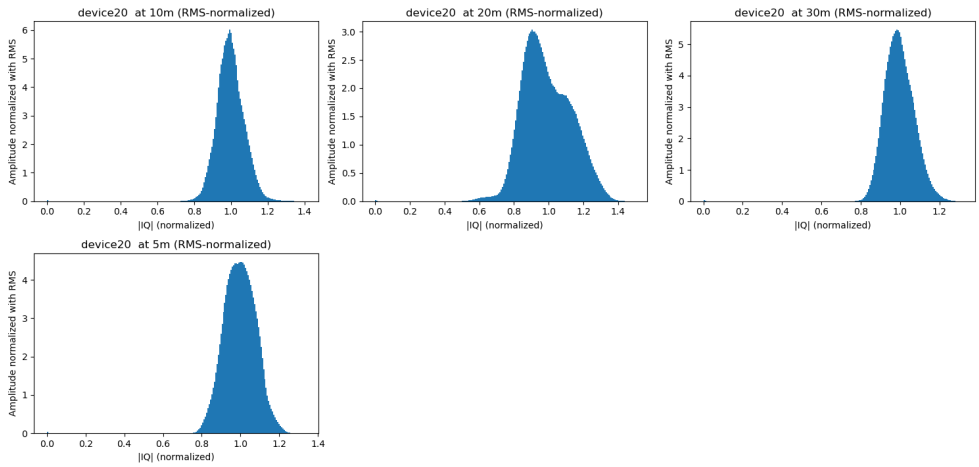


Figure 12: Histograms of the 20th device by distance indoors. Plotting the normalized amplitude with respect to the IQ. Ideal distribution is a peak at 1 in the X axis.

1) For example it might be possible to use a single laptop as the receiver and transmitter and use USB extension to connect to the SDR with longer distances.
2) A Ethernet cable might be more appropriate and faster.
3) Fine-tuning the different parameters such as the 0-padding or change the sampling frequency to $f - s = 2 * BW$ as suggested a recent article [3].

References

[1] Aqeel Ahmed et al. "A Comprehensive Survey on Deep Learning-Based LoRa Radio Frequency Fingerprinting Identification". In: *Sensors* 24.13 (2024). ISSN: 1424-8220. DOI: `10.3390/s24134411`. URL: `https://www.mdpi.com/1424-8220/24/13/4411`.

[2] Maxime Bazin. *2024 implementation*. 2024. URL: `https://github.com/kazawai`.

[3] Alireza Maleki et al. "A Tutorial on Chirp Spread Spectrum Modulation for LoRaWAN: Basics and Key Advances". In: *IEEE Open Journal of the Communications Society* 5 (2024), pp. 4578–4612. DOI: `10.1109/OJCOMS.2024.3433502`.

[4] Amani Al-Shawabka et al. "DeepLoRa: Fingerprinting LoRa devices at scale through deep learning and data augmentation". In: *Proceedings of the Twenty-second International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 2021, pp. 251–260.

[5] Guanxiong Shen et al. "Radio frequency fingerprint identification for LoRa using spectrogram and CNN". In: *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE. 2021, pp. 1–10.

[6] Zhenqiang Xu et al. "From Demodulation to Decoding: Toward Complete LoRa PHY Understanding and Implementation". In: *ACM Trans. Sen. Netw.* 18.4 (Jan. 2023). ISSN: 1550-4859. DOI: 10.1145/3546869. URL: https://doi.org/10.1145/3546869.

Appendices

A First appendix

B Second appendix

Notes

photo of the Receiver part in appendix 7
photo of the transmitter part in appendix 8
add a reference for the polarization effect 8
Add photos of the layout in appendix 8
footnote or citation ? 11